

CHAPTER 1

INTRODUCTION

1.1 Introduction

The stock market has become one of the most significant investment platforms in the modern financial world. Every year, millions of people begin investing in stocks to build wealth and achieve financial independence. However, beginners often hesitate to participate in the stock market due to the high financial risks associated with real-money trading. A lack of practical knowledge, market understanding, and trading experience often results in poor investment decisions and financial losses.

To address these challenges, paper trading platforms have become increasingly popular. Paper trading allows users to simulate buying and selling stocks using virtual money while tracking real-time market prices. This enables users to understand stock market operations, develop investment strategies, and gain practical experience without risking actual capital.

SB Stocks – Stock Portfolio Management System is a web-based paper trading platform developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js). The platform provides users with a realistic stock trading environment where they can register securely, manage virtual funds, search stocks, buy and sell shares, and monitor their investment portfolios using live stock market data obtained through the Alpha Vantage API.

The application integrates secure authentication using JSON Web Tokens (JWT), ensuring that user accounts and trading data remain protected. Users can view detailed information about stocks, maintain transaction histories, monitor portfolio performance, and evaluate investment decisions through an intuitive and user-friendly dashboard.

The project aims to bridge the gap between theoretical financial knowledge and practical stock trading experience. By providing a safe and interactive simulation environment, SB Stocks enables students, beginners, and aspiring investors to build confidence before participating in actual financial markets.

1.2 Background of the Project

Stock trading traditionally requires investors to use real money, making every investment decision financially significant. Many beginners lack sufficient knowledge regarding stock selection, portfolio diversification, risk management, and market behavior. Consequently, they often incur substantial financial losses due to inexperience.

Paper trading systems provide an effective solution by allowing users to perform simulated trading activities using virtual currency while tracking actual market prices. Such platforms serve as valuable educational tools for students, finance enthusiasts, and first-time investors.

SB Stocks has been developed to offer an accessible, secure, and realistic trading environment where users can learn market fundamentals, practice investment strategies, and improve financial decision-making skills. The platform combines modern web technologies with real-time financial APIs to provide an engaging and educational experience.

1.3 Problem Statement

Learning stock market trading through theoretical concepts alone is insufficient for developing practical investment skills. Existing stock trading platforms require users to invest real money, making it difficult for beginners to experiment without financial risk. Furthermore, many educational platforms lack realistic trading simulations, secure portfolio management, and real-time market integration.

Therefore, there is a need for an interactive paper trading platform that enables users to practice stock trading using virtual funds while accessing real-time stock market data in a secure and user-friendly environment.

1.4 Objectives

The primary objectives of the SB Stocks project are:

- To develop a secure web-based stock portfolio management system.
 - To simulate stock trading using virtual currency.
 - To provide real-time stock market information using the Alpha Vantage API.
 - To allow users to buy and sell stocks without financial risk.
 - To maintain detailed transaction histories for every user.
 - To display portfolio performance through an interactive dashboard.
 - To implement secure authentication using JWT.
 - To improve users' understanding of investment strategies and stock market operations.
 - To provide an educational platform for beginners and students.
 - To demonstrate full-stack web application development using the MERN Stack.
-

1.5 Scope of the Project

The scope of SB Stocks focuses on providing a realistic paper trading experience rather than facilitating actual financial transactions. The application enables users to simulate investment decisions using virtual money while tracking live stock prices.

The system includes user registration, login authentication, stock searching, buying and selling of shares, portfolio management, transaction history, and account management. Administrators are not required because each user independently manages their own virtual portfolio.

Future versions of the application can include advanced analytical tools, AI-based investment recommendations, stock news integration, technical indicators, watchlists, mobile application support, and multi-user competitions.

1.6 Motivation

The motivation behind developing SB Stocks stems from the increasing importance of financial literacy among students and young professionals. Many individuals wish to learn investing but fear losing money during the learning process.

By providing a realistic simulation platform, users can gain practical exposure to stock market concepts without financial consequences. Additionally, the project demonstrates the integration of modern web technologies, RESTful APIs, secure authentication mechanisms, and cloud databases, making it an excellent academic and professional project.

1.7 Applications

SB Stocks can be used in several domains, including:

- Educational institutions for finance and investment courses.

- Universities for project demonstrations and research.
 - Beginner investors learning stock market concepts.
 - Investment clubs for training purposes.
 - Financial literacy workshops.
 - Online learning platforms.
 - Career preparation for finance and fintech professionals.
 - Software engineering portfolio demonstrations.
-

1.8 Advantages of the Proposed System

The proposed system provides several advantages over traditional learning methods:

- Risk-free stock trading simulation.
 - Real-time stock market information.
 - Secure user authentication.
 - Interactive portfolio dashboard.
 - Easy-to-use user interface.
 - Complete transaction history.
 - Cloud-based database management.
 - Responsive web application.
 - Fast API communication.
 - Practical learning environment.
-

1.9 Technology Stack

Layer	Technology Used
Frontend	React.js, HTML5, CSS3, JavaScript
Backend	Node.js, Express.js
Database	MongoDB Atlas
Authentication	JSON Web Token (JWT), bcrypt
API	Alpha Vantage API
Version Control	Git & GitHub
Deployment	Vercel (Frontend), Render (Backend)
Development Tools VS Code, Postman	

1.10 Organization of the Report

This project report is organized into multiple chapters to provide a systematic understanding of the developed application.

- **Chapter 1** introduces the project, objectives, motivation, scope, and technologies used.
- **Chapter 2** presents the literature survey, existing systems, related research, and comparative analysis.
- **Chapter 3** explains the requirement analysis, functional and non-functional requirements, and feasibility study.
- **Chapter 4** discusses the system design, architecture, database schema, UML diagrams, and API design.
- **Chapter 5** describes the implementation details of the frontend, backend, authentication, and database integration.

- **Chapter 6** explains the testing methodology, test cases, and validation procedures.
- **Chapter 7** presents project results, screenshots, and performance evaluation.
- **Chapter 8** concludes the report and discusses future enhancements.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

A literature survey is an essential part of software project development that involves studying existing research papers, software applications, and technologies related to the proposed system. It helps identify the strengths and limitations of current solutions while providing a foundation for designing a better system.

For the **SB Stocks – Stock Portfolio Management System**, various research papers, online trading platforms, paper trading applications, and portfolio management systems were studied. The survey focuses on stock market prediction, portfolio management, paper trading systems, secure web applications, and financial technology platforms.

The study revealed that although many commercial trading platforms provide real-time trading facilities, very few offer a complete learning environment for beginners without involving financial risk. This motivated the development of a secure paper trading application that combines real-time stock information with virtual trading.

2.2 Review of Existing Systems

Several stock trading applications are currently available in the market. These platforms mainly focus on real-money investments and advanced trading features.

Some popular trading platforms include:

- Zerodha Kite
- Groww
- Upstox
- TradingView
- Investopedia Stock Simulator
- Yahoo Finance
- Moneybhai Virtual Trading

Although these applications provide useful services, they also present certain limitations for beginners.

Common limitations include:

- Real-money investment requirement
- Limited educational support
- Complex trading interfaces
- Subscription-based premium features
- Limited customization
- Lack of beginner-friendly portfolio simulations

These limitations create the need for an educational stock trading simulator that allows users to practice investing without financial risk.

2.3 Research Paper 1

Title

Stock Market Prediction Using Machine Learning Algorithms

Authors

Patel J., Shah S., Thakkar P., Kotecha K.

Summary

The paper discusses the application of machine learning algorithms for predicting stock prices using historical market data. Different algorithms such as Decision Trees, Support Vector Machines, and Random Forest were evaluated to improve prediction accuracy.

Findings

- Machine learning improves prediction accuracy.
- Historical data plays an important role.
- Multiple features increase model performance.

Limitation

Prediction accuracy decreases during sudden market fluctuations.

2.4 Research Paper 2

Title

Portfolio Management Using Modern Web Technologies

Summary

The paper presents a web-based portfolio management system that enables investors to monitor investments through dashboards and data visualization.

Findings

- Interactive dashboards improve user experience.
- Secure authentication protects financial information.
- Cloud databases improve scalability.

Limitation

No paper trading functionality.

2.5 Research Paper 3

Title

Paper Trading Systems for Financial Education

Summary

This paper explains how virtual trading platforms help beginners understand investment strategies before entering real markets.

Findings

- Risk-free learning environment.
- Improved investment confidence.
- Better financial literacy.

Limitation

Limited real-time market integration.

2.6 Research Paper 4

Title

RESTful APIs for Financial Applications

Summary

The research discusses integration of third-party financial APIs into web applications using REST architecture.

Findings

- Faster communication
- Platform independence
- Scalable architecture
- Secure API communication

Limitation

Dependent on API availability.

2.7 Research Paper 5

Title

Secure Authentication Using JSON Web Tokens

Summary

This paper explains secure authentication mechanisms using JWT for web applications.

Findings

- Stateless authentication
- Better security
- Easy implementation
- High scalability

Limitation

Requires proper token expiration handling.

2.8 Comparative Analysis

Feature	Existing Systems SB Stocks	
Paper Trading	Partial	✓ Yes
Virtual Portfolio	Limited	✓ Yes
Real-Time Stock Data	✓ Yes	✓ Yes
Secure Login	✓ Yes	✓ JWT Authentication
Transaction History	✓ Yes	✓ Yes
Beginner Friendly	Moderate	✓ Excellent
Free Learning Platform	Partial	✓ Yes
Portfolio Dashboard	Basic	✓ Interactive

Feature	Existing Systems SB Stocks	
Cloud Database	Partial	✓ MongoDB Atlas
Open Source	No	✓ Yes

2.9 Research Gap

After studying existing applications and research papers, the following research gaps were identified:

- Most trading platforms require real investments.
- Few applications focus on educational learning.
- Portfolio simulators often use outdated market data.
- Many systems have complicated interfaces.
- Limited support for beginner investors.
- Few open-source paper trading systems exist.
- Some systems lack secure authentication.
- Integration of modern MERN technologies is limited.
- Personalized learning features are absent.
- AI-assisted investment recommendations are generally unavailable.

The proposed **SB Stocks** system addresses these gaps by providing a secure, educational, user-friendly paper trading platform with real-time market information and portfolio management capabilities.

2.10 Advantages Over Existing Systems

The proposed system offers several improvements over existing solutions:

- Completely risk-free trading environment.
- Real-time stock information using Alpha Vantage API.
- Secure authentication with JWT.
- Interactive dashboard for portfolio monitoring.
- Fast and responsive React.js interface.
- Cloud-hosted MongoDB database.
- Easy navigation for beginners.
- Virtual buying and selling of stocks.
- Complete transaction history.
- Expandable architecture for future enhancements.

2.11 Conclusion

The literature survey demonstrates that while existing stock trading platforms provide comprehensive investment tools, they often fail to meet the educational needs of beginners. Most commercial applications prioritize real-money trading rather than learning and experimentation.

The proposed **SB Stocks – Stock Portfolio Management System** addresses these limitations by combining real-time market information, secure authentication, virtual trading, and intuitive portfolio management into a single web application. The use of the MERN stack ensures scalability, responsiveness, and maintainability, making the system suitable for educational institutions, students, and aspiring investors.

References (IEEE Style)

1. J. Patel, S. Shah, P. Thakkar, and K. Kotecha, "Predicting Stock and Stock Price Index Movement Using Trend Deterministic Data Preparation and Machine Learning Techniques," *Expert Systems with Applications*, vol. 42, no. 1, pp. 259–268.
2. E. F. Fama, "Efficient Capital Markets: A Review of Theory and Empirical Work," *Journal of Finance*, vol. 25, no. 2, pp. 383–417.
3. Alpha Vantage Inc., *Alpha Vantage API Documentation*. Available: <https://www.alphavantage.co/documentation/>
4. MongoDB Inc., *MongoDB Atlas Documentation*. Available: <https://www.mongodb.com/docs/atlas/>
5. React Team, *React Documentation*. Available: <https://react.dev/>
6. Node.js Foundation, *Node.js Documentation*. Available: <https://nodejs.org/docs>
7. Express.js Team, *Express Framework Documentation*. Available: <https://expressjs.com/>
8. OWASP Foundation, *JSON Web Token Best Practices*. Available: <https://owasp.org/>

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Introduction

Requirement Analysis is one of the most important phases in the Software Development Life Cycle (SDLC). It involves identifying, documenting, analyzing, and validating the functional and non-functional requirements of the proposed system. Proper requirement analysis ensures that the software meets user expectations while minimizing development risks.

For the **SB Stocks – Stock Portfolio Management System**, this phase focused on understanding the needs of beginner investors, students, and users who want to practice stock trading without financial risk. The analysis also identified the technical requirements necessary for implementing a secure, scalable, and user-friendly web application using the MERN Stack.

The project requirements were gathered by studying existing stock trading platforms, paper trading applications, financial technology websites, and user expectations regarding portfolio management systems.

3.2 Software Requirement Specification (SRS)

The Software Requirement Specification (SRS) defines the complete functionality of the SB Stocks application. It serves as a blueprint for the design, implementation, testing, and maintenance phases.

The SRS document ensures that developers, evaluators, and users have a common understanding of the project's objectives and functionalities.

3.3 Functional Requirements

Functional requirements describe what the system should do.

FR-1 User Registration

The system shall allow new users to create an account using:

- Full Name
- Email Address
- Password

The password will be securely encrypted using bcrypt before storing it in the database.

FR-2 User Login

The system shall authenticate registered users using:

- Email
- Password

After successful authentication, a JWT token shall be generated for secure session management.

FR-3 User Authentication

The application shall protect all private routes using JWT Authentication.

Only authenticated users shall access:

- Dashboard
- Portfolio
- Transactions
- Profile
- Buy/Sell Operations

FR-4 Dashboard

The dashboard shall display:

- Total Portfolio Value
- Available Virtual Balance
- Number of Stocks Owned
- Recent Transactions
- Portfolio Summary

FR-5 Stock Search

Users shall search stocks using:

- Company Name
- Stock Symbol

The application shall retrieve real-time stock data from Alpha Vantage API.

FR-6 Buy Stocks

The application shall allow users to purchase stocks using virtual money.

The system shall verify:

- Available balance
- Stock availability
- Quantity entered

After successful purchase:

- Portfolio updates
 - Balance updates
 - Transaction stored
-

FR-7 Sell Stocks

Users shall sell previously purchased stocks.

The system shall verify:

- Ownership
- Quantity available

After selling:

- Portfolio updated
- Balance updated

- Transaction recorded
-

FR-8 Portfolio Management

Users shall view:

- Current Holdings
 - Purchase Price
 - Current Price
 - Profit/Loss
 - Quantity Owned
-

FR-9 Transaction History

The application shall maintain complete records of:

- Buy Transactions
- Sell Transactions

Each record includes:

- Stock Name
 - Quantity
 - Price
 - Date
 - Transaction Type
-

FR-10 Profile Management

Users shall update:

- Name
 - Email
 - Password
-

FR-11 Logout

Users shall securely logout from the application by deleting the authentication token.

3.4 Non-Functional Requirements

The non-functional requirements describe how the system performs.

Performance

- Fast page loading
 - API response within acceptable time
 - Efficient database queries
-

Security

- Password Encryption
 - JWT Authentication
 - Secure REST APIs
 - Input Validation
 - Protected Routes
-

Reliability

The application should be available whenever users access it.

Proper error handling must prevent application crashes.

Scalability

The system should support thousands of users without significant performance degradation.

MongoDB Atlas provides scalable cloud database support.

Availability

The application shall remain available 24×7 after deployment.

Maintainability

The project follows modular architecture.

Frontend and backend are developed independently for easier maintenance.

Portability

The system runs on:

- Windows
- Linux
- macOS

Supported browsers:

- Chrome
 - Firefox
 - Edge
 - Brave
-

Usability

The application provides:

- Simple Interface
 - Responsive Design
 - Easy Navigation
 - Beginner-Friendly Dashboard
-

3.5 User Requirements

The end users expect the system to:

- Register securely
- Login easily
- Search stocks
- Buy stocks
- Sell stocks
- Monitor portfolio
- View transaction history
- Logout securely

3.6 System Requirements

Software Requirements

Software	Version
Operating System	Windows 10/11
VS Code	Latest
Node.js	v18+
React.js	Latest
Express.js	Latest
MongoDB Atlas	Cloud
Git	Latest
GitHub	Cloud Repository
Postman	Latest
Chrome Browser	Latest

Hardware Requirements

Component Specification

Processor	Intel Core i3 or Higher
RAM	Minimum 4 GB
Hard Disk	20 GB Free Space

Component Specification

Internet Broadband Connection

Display 1366 × 768 or Higher

3.7 Feasibility Study

Before implementation, the feasibility of the project was analyzed.

Technical Feasibility

The proposed system is technically feasible because all technologies used are open-source and widely adopted.

Technologies include:

- React.js
- Node.js
- Express.js
- MongoDB Atlas
- JWT
- Alpha Vantage API

These technologies provide scalability, flexibility, and community support.

Economic Feasibility

The project requires minimal financial investment.

Most software used is free.

Open-source technologies significantly reduce development costs.

Therefore, the project is economically feasible.

Operational Feasibility

The application is simple to operate.

Users require only basic computer knowledge.

The interface is designed for beginners.

Hence, the system is operationally feasible.

Schedule Feasibility

The project can be completed within the allotted SmartBridge internship duration.

Proper planning ensures timely completion of all phases.

3.8 Assumptions

The following assumptions were considered during development:

- Internet connection is available.
 - Alpha Vantage API remains operational.
 - MongoDB Atlas is accessible.
 - Users possess basic computer knowledge.
 - Browsers support modern JavaScript.
-

3.9 Constraints

The project has the following limitations:

- Uses free Alpha Vantage API requests.
 - Virtual trading only.
 - No real money transactions.
 - Internet required.
 - Dependent on third-party API availability.
-

3.10 Actors

The system includes one primary actor.

User

Responsibilities:

- Register
 - Login
 - Search Stocks
 - Buy Stocks
 - Sell Stocks
 - Manage Portfolio
 - View Transactions
 - Update Profile
 - Logout
-

3.11 Use Case Description

Register

Actor: User

Description: Creates a new account.

Precondition: User is not registered.

Postcondition: Account created successfully.

Login

Actor: User

Description: Authenticates the user.

Precondition: Registered account.

Postcondition: Dashboard displayed.

Buy Stock

Actor: User

Description: Purchases stock using virtual money.

Precondition: Logged in.

Postcondition: Portfolio updated.

Sell Stock

Actor: User

Description: Sells owned stocks.

Precondition: Stock available.

Postcondition: Portfolio updated.

View Portfolio

Actor: User

Description: Displays all owned stocks.

View Transactions

Actor: User

Description: Shows complete trading history.

3.12 Requirement Traceability Matrix (RTM)

Requirement ID	Description	Module
FR1	User Registration	Authentication
FR2	Login	Authentication
FR3	JWT Security	Security
FR4	Dashboard	Dashboard
FR5	Search Stocks	Stock Module
FR6	Buy Stocks	Trading Module
FR7	Sell Stocks	Trading Module
FR8	Portfolio	Portfolio Module
FR9	Transaction History	Transaction Module

Requirement ID Description		Module
FR10	Profile	User Module
FR11	Logout	Authentication

3.13 Chapter Summary

The Requirement Analysis phase identified all functional and non-functional requirements necessary for developing the **SB Stocks – Stock Portfolio Management System**. The analysis confirmed that the proposed application is technically, economically, operationally, and schedule feasible. These requirements serve as the foundation for the subsequent design and implementation phases of the project.

CHAPTER 4

SYSTEM DESIGN

4.1 Introduction

System Design is the process of transforming the software requirements into a structured blueprint that guides the implementation of the

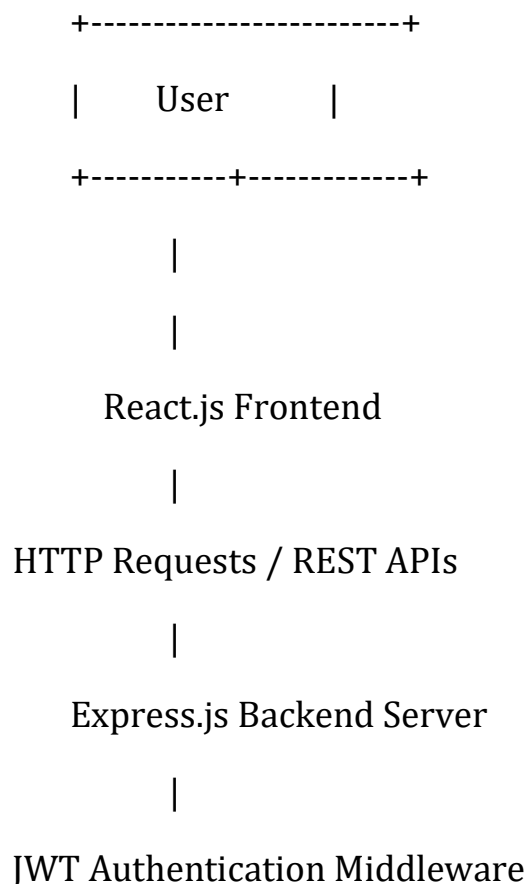
application. It defines the architecture, database structure, user interactions, and communication between different software components.

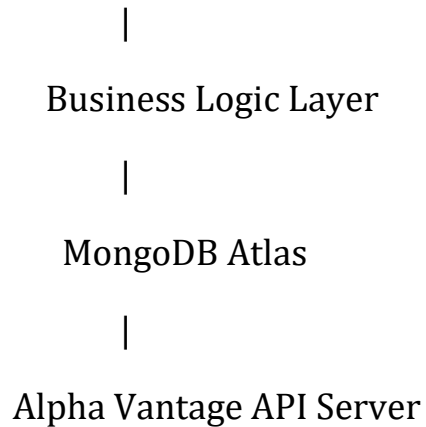
For the **SB Stocks – Stock Portfolio Management System**, the design follows a modular architecture based on the MERN Stack (MongoDB, Express.js, React.js, and Node.js). The application adopts RESTful APIs for communication between the frontend and backend while using MongoDB Atlas as the cloud database.

The system has been designed to ensure scalability, maintainability, security, and high performance.

4.2 System Architecture

The application follows a three-tier architecture.





Architecture Description

The frontend developed in React.js provides an interactive user interface. Users perform actions such as login, stock search, buying stocks, and viewing portfolios.

The backend developed using Express.js receives requests, validates user credentials, performs business logic, and communicates with the MongoDB Atlas database.

The Alpha Vantage API provides real-time stock market information.

4.3 Modules of the System

The project consists of the following modules.

Module 1: User Authentication

Functions

- Register User
- Login User
- Password Encryption
- JWT Token Generation
- Logout

Purpose

Provides secure authentication and protects user data.

Module 2: Dashboard

Functions

- Portfolio Summary
- Current Balance
- Portfolio Value
- Recent Transactions
- Performance Overview

Purpose

Provides users with an overview of their investments.

Module 3: Stock Search

Functions

- Search Company
- Search Symbol
- Display Stock Price
- Market Information

Purpose

Allows users to access live stock information.

Module 4: Trading Module

Functions

- Buy Stocks
- Sell Stocks
- Quantity Validation
- Balance Validation

Purpose

Simulates stock trading using virtual funds.

Module 5: Portfolio Management

Functions

- Display Holdings
- Profit/Loss Calculation
- Current Value
- Total Investment

Purpose

Tracks user investments.

Module 6: Transaction History

Functions

- Buy History
- Sell History

- Date and Time
- Transaction Status

Purpose

Maintains a complete audit trail of user activities.

Module 7: Profile Management

Functions

- Edit Profile
- Change Password
- Update User Details

Purpose

Allows users to manage personal information.

4.4 Data Flow Diagram (DFD)

Level 0 DFD

+-----+

| User |

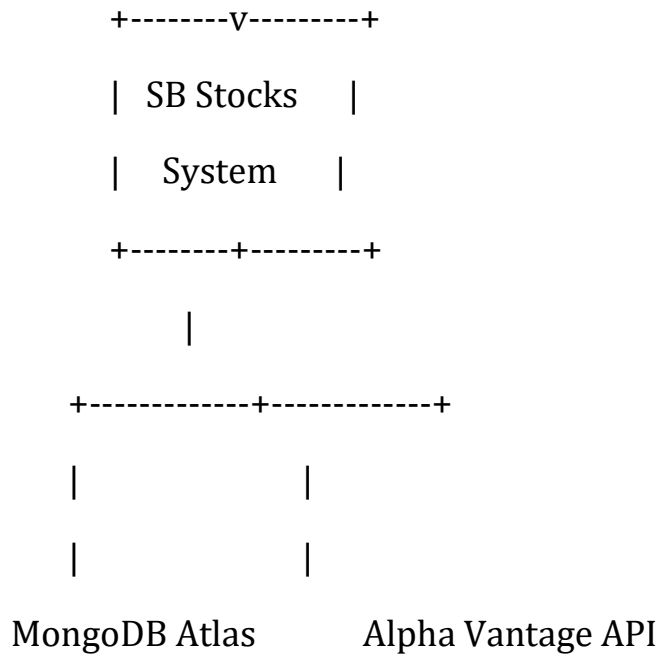
+-----+

|

|

Requests/Responses

|



Level 1 DFD

User



Register/Login



Authentication



Dashboard



Search Stock



Buy/Sell Stock



Portfolio

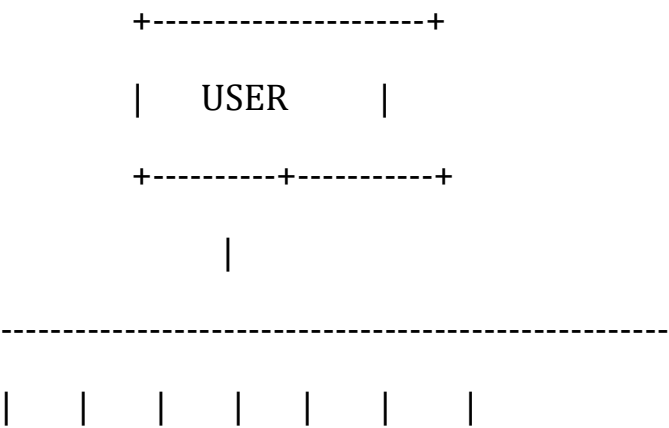


Transaction History



Logout

4.5 Use Case Diagram



Use Case Description

Use Case	Description
Register	Creates new account
Login	Authenticates user
Search Stock	Displays stock details
Buy Stock	Purchases stock
Sell Stock	Sells stock
Portfolio	Displays holdings
Transactions	Shows trading history
Logout	Ends session

4.6 Activity Diagram

Start



Open Website



Login/Register



Authentication Successful?



Yes



Dashboard



Search Stock



Buy or Sell



Update Portfolio



View Transactions



Logout



End

4.7 Sequence Diagram

User



Frontend



Backend API



Authentication



MongoDB



Return Response

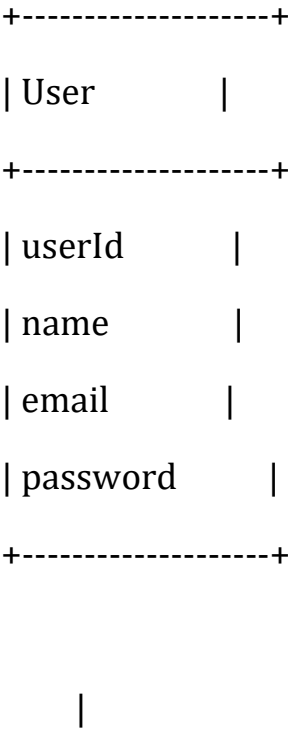


Frontend



Display Dashboard

4.8 Class Diagram



|

+-----+

| Portfolio |

+-----+

| portfolioId |

| stockName |

| quantity |

| purchasePrice |

| currentPrice |

+-----+

|

|

+-----+

| Transaction |

+-----+

| transactionId |

| stockName |

buy/sell	
quantity	
date	
+-----+	

4.9 Entity Relationship Diagram (ER Diagram)

User

User_ID (PK)

Name

Email

Password

|

|

Portfolio

Portfolio_ID (PK)

User_ID (FK)

Stock_Name

Quantity

Purchase_Price

|

|

Transaction

Transaction_ID (PK)

Portfolio_ID (FK)

Type

Price

Quantity

Date

4.10 Database Design

Collection: Users

Field	Type
_id	ObjectId
name	String
email	String

Field	Type
-------	------

password	String
----------	--------

Collection: Portfolio

Field	Type
-------	------

_id	ObjectId
-----	----------

userId	ObjectId
--------	----------

stockName	String
-----------	--------

quantity	Number
----------	--------

purchasePrice	Number
---------------	--------

Collection: Transactions

Field	Type
-------	------

_id	ObjectId
-----	----------

userId	ObjectId
--------	----------

stockName	String
-----------	--------

quantity	Number
----------	--------

price	Number
-------	--------

type	String
------	--------

date	Date
------	------

4.11 API Design

Method	Endpoint	Description
POST	/api/auth/register	Register User
POST	/api/auth/login	Login User
GET	/api/stocks/search	Search Stocks
POST	/api/portfolio/buy	Buy Stock
POST	/api/portfolio/sell	Sell Stock
GET	/api/portfolio	View Portfolio
GET	/api/transactions	View Transactions
PUT	/api/profile	Update Profile

4.12 Folder Structure

SB-STOCKS

|

└─ client

|

└─ src

|

└─ components

| └─ pages

| └─ services

| └─ hooks

| └─ App.jsx

|

└─ backend

| └─ controllers

| └─ routes

| └─ middleware

| └─ models

| └─ config

| └─ utils

| └─ server.js

|

└─ package.json

└─ README.md

└─ .env

4.13 User Interface Design

The application consists of the following interfaces:

- Home Page
- Login Page
- Registration Page
- Dashboard
- Stock Search Page
- Portfolio Page
- Buy Stock Page

- Sell Stock Page
- Transaction History Page
- User Profile Page

The UI follows a modern responsive design using React.js components, ensuring compatibility across desktops, tablets, and mobile devices.

4.14 Security Design

The system incorporates several security mechanisms:

- Password encryption using **bcrypt**
 - JWT-based authentication
 - Protected API routes
 - Input validation and sanitization
 - Environment variables for API keys and secrets
 - Secure communication between client and server
-

4.15 Chapter Summary

The system design provides a detailed blueprint of the **SB Stocks – Stock Portfolio Management System**. The modular architecture, database schema, API design, and UML diagrams collectively ensure that the application is scalable, secure, and maintainable. These design artifacts serve as the foundation for the implementation phase discussed in the next chapter.

CHAPTER 5

PROJECT DEVELOPMENT AND IMPLEMENTATION

5.1 Introduction

The implementation phase transforms the system design into a fully functional software application. During this phase, the frontend, backend, database, authentication mechanism, and external APIs were integrated to develop the **SB Stocks – Stock Portfolio Management System**.

The project follows the MERN (MongoDB, Express.js, React.js, and Node.js) architecture, where the frontend provides an interactive user interface, the backend handles business logic, MongoDB Atlas stores application data, and the Alpha Vantage API supplies real-time stock information.

The application was developed in a modular manner to improve readability, maintainability, and scalability.

5.2 Development Methodology

The project was developed following the **Agile Software Development Methodology**, which emphasizes iterative development and continuous testing.

The development process consisted of the following stages:

1. Requirement Gathering
2. System Analysis
3. UI/UX Design
4. Backend Development

5. Database Integration
6. API Integration
7. Testing
8. Deployment

The Agile approach enabled incremental development and regular improvements based on testing and feedback.

5.3 Frontend Development

The frontend was developed using **React.js**, providing a responsive and interactive user interface.

Technologies Used

- React.js
- JavaScript (ES6+)
- HTML5
- CSS3
- React Router DOM
- Axios

Main Components

- Home Page
- Login Page
- Registration Page
- Dashboard
- Search Stocks

- Portfolio
- Transaction History
- Profile Page
- Navigation Bar

React's component-based architecture promotes code reusability and simplifies application maintenance.

5.4 Backend Development

The backend was developed using **Node.js** and **Express.js**, which provide a fast and scalable environment for building RESTful APIs.

Backend Responsibilities

- User authentication
- Portfolio management
- Transaction processing
- Stock data retrieval
- Database operations
- Error handling

Backend Structure

backend/

|

└─ config/

└─ controllers/

└─ middleware/

└─ models/

└─ routes/

└─ utils/

└─ server.js

└─ package.json

Each folder performs a specific responsibility, improving maintainability.

5.5 Database Development

MongoDB Atlas was selected as the cloud database because of its scalability, flexibility, and document-oriented architecture.

Collections

Users

Stores user information.

Fields:

- _id
 - name
 - email
 - password
-

Portfolio

Stores purchased stocks.

Fields:

- userId
 - stockSymbol
 - companyName
 - quantity
 - purchasePrice
 - currentPrice
-

Transactions

Stores trading history.

Fields:

- userId
- stockSymbol
- quantity

- price
 - transactionType
 - timestamp
-

5.6 Authentication Module

User authentication is implemented using **JWT (JSON Web Token)**.

Registration Process

1. User enters details.
2. Password encrypted using bcrypt.
3. Data stored in MongoDB.
4. Success message displayed.

Login Process

1. Email verification.
2. Password verification.
3. JWT generated.
4. Token sent to frontend.
5. Protected routes unlocked.

Security Features

- Password hashing
- JWT authentication
- Route protection
- Token expiration

- Unauthorized access prevention
-

5.7 Stock Search Module

The Stock Search Module enables users to retrieve stock information.

Features

- Search by company name
- Search by stock symbol
- Current market price
- Company overview
- Latest market updates

The module communicates with the Alpha Vantage API using REST requests.

5.8 Portfolio Module

The Portfolio Module allows users to monitor their investments.

Displayed Information

- Company Name
- Stock Symbol
- Quantity
- Purchase Price
- Current Price
- Total Investment
- Profit/Loss

The module automatically updates whenever a user buys or sells stocks.

5.9 Trading Module

This module simulates real stock trading.

Buy Stock Workflow

1. User selects stock.
2. Enters quantity.
3. System validates balance.
4. Portfolio updated.
5. Transaction saved.
6. Dashboard refreshed.

Sell Stock Workflow

1. User selects owned stock.
 2. Quantity verified.
 3. Shares deducted.
 4. Virtual balance updated.
 5. Transaction stored.
-

5.10 Dashboard Module

The Dashboard is the central page of the application.

It displays:

- Available Balance

- Portfolio Value
- Number of Stocks Owned
- Recent Transactions
- Profit/Loss Summary

The dashboard provides users with a quick overview of their portfolio performance.

5.11 Transaction Module

Every trading activity is recorded.

Information stored:

- Transaction ID
- User ID
- Stock Name
- Quantity
- Buy/Sell
- Price
- Date & Time

Maintaining transaction history improves transparency and allows users to review their trading activities.

5.12 API Integration

Real-time stock information is retrieved using the **Alpha Vantage API**.

Advantages

- Live stock prices
- Company details
- Market updates
- Reliable REST API
- JSON responses

The frontend sends API requests through the backend, which processes the responses and returns the required data to the client.

5.13 Application Workflow

User



React Frontend



Express REST API



JWT Authentication

↓

MongoDB Atlas

↓

Business Logic

↓

Alpha Vantage API

↓

Response

↓

Dashboard Updated

5.14 Error Handling

The application includes robust error handling.

Examples:

- Invalid login credentials
- Duplicate email registration
- Stock not found
- Insufficient virtual balance
- Invalid quantity
- Database connection errors
- API request failures
- Network issues

Meaningful error messages are displayed to improve the user experience.

5.15 Deployment

The application was deployed using modern cloud platforms.

Frontend

- Vercel

Backend

- Render

Database

- MongoDB Atlas

This architecture ensures scalability, high availability, and easy maintenance.

5.16 Project Folder Structure

SB-STOCKS

|

└─ client

|

└─ public

|

└─ src

|

|

└─ components

|

|

└─ pages

|

|

└─ services

|

|

└─ assets

|

|

└─ hooks

|

|

└─ App.jsx

|

└─ backend

|

└─ config

|

└─ controllers

|

└─ middleware

|

└─ models

|

└─ routes

|

└─ utils

|

└─ server.js

|

└─ package.json

|

└── README.md

└── .gitignore

└── package.json

5.17 Advantages of the Developed System

The developed application provides several advantages:

- User-friendly interface
- Secure authentication
- Real-time stock data
- Fast performance
- Modular architecture
- Responsive design
- Cloud database integration
- Risk-free paper trading
- Easy maintenance
- Scalable design

5.18 Challenges Faced During Development

The following challenges were encountered during implementation:

- Configuring JWT authentication securely.
- Integrating Alpha Vantage API within free request limits.
- Managing asynchronous API requests.
- Handling MongoDB Atlas cloud connectivity.
- Maintaining synchronized portfolio updates after transactions.
- Ensuring responsive UI across different devices.
- Deploying frontend and backend on separate cloud platforms.

These challenges were addressed through testing, debugging, and modular development practices.

5.19 Chapter Summary

The implementation phase successfully transformed the proposed design into a fully functional web application. The use of the MERN Stack enabled the development of a secure, scalable, and responsive platform for virtual stock trading. Integration with MongoDB Atlas, JWT authentication, and the Alpha Vantage API ensured real-time functionality and secure user management. The modular architecture facilitates future enhancements and maintenance.

CHAPTER 6

SYSTEM TESTING

6.1 Introduction

Testing is a critical phase in the Software Development Life Cycle (SDLC). It ensures that the developed application functions according to the specified requirements and is free from defects. The primary objective of testing is to identify and resolve errors before deployment, thereby improving the reliability, security, and performance of the software.

For the **SB Stocks – Stock Portfolio Management System**, extensive testing was performed on the frontend, backend, database interactions, authentication module, and stock trading functionalities. Various testing techniques such as Unit Testing, Integration Testing, System Testing, and User Acceptance Testing (UAT) were employed to verify the correctness of the application.

6.2 Objectives of Testing

The main objectives of testing are:

- Verify that all functional requirements are satisfied.
 - Ensure secure user authentication.
 - Validate stock buying and selling operations.
 - Confirm correct portfolio calculations.
 - Detect and eliminate software defects.
 - Ensure database consistency.
 - Verify API integration.
 - Improve overall software quality.
-

6.3 Testing Strategy

The testing process was divided into several stages:

- Unit Testing
- Integration Testing
- System Testing
- Functional Testing
- Performance Testing
- Security Testing
- User Acceptance Testing

Each testing stage focused on validating different components of the application.

6.4 Types of Testing

6.4.1 Unit Testing

Unit Testing verifies individual modules independently.

Modules Tested:

- Login Module
- Registration Module
- Portfolio Module
- Stock Search Module
- Buy Stock Module
- Sell Stock Module
- Transaction Module

Result: All individual modules functioned correctly.

6.4.2 Integration Testing

Integration Testing verifies communication between modules.

Modules Integrated:

- React Frontend ↔ Express Backend
- Backend ↔ MongoDB Atlas
- Backend ↔ Alpha Vantage API
- Authentication ↔ Protected Routes

Result: Data was exchanged successfully without errors.

6.4.3 System Testing

System Testing evaluates the application as a complete system.

Verified Features:

- User Registration
- User Login
- Dashboard
- Portfolio
- Buy Stocks
- Sell Stocks
- Transaction History
- Logout

Result: The system functioned as expected.

6.4.4 Functional Testing

Functional Testing verifies that every feature satisfies the functional requirements.

Verified Functions:

- User Authentication
- JWT Token Generation
- Stock Search
- Portfolio Updates
- Transaction Recording
- Profile Update

Result: All functions passed successfully.

6.4.5 Performance Testing

Performance testing was performed to ensure that the application remained responsive under normal usage conditions.

Observations:

- Fast page loading
 - Smooth navigation
 - Quick API responses
 - Efficient database queries
-

6.4.6 Security Testing

Security measures tested include:

- Password Encryption
- JWT Authentication
- Protected Routes
- Unauthorized Access Prevention
- Input Validation

Result: The application successfully prevented unauthorized access.

6.5 Test Environment

Component	Specification
Operating System	Windows 11
Browser	Google Chrome
Frontend	React.js
Backend	Node.js & Express.js
Database	MongoDB Atlas
API	Alpha Vantage
IDE	Visual Studio Code
API Testing Tool	Postman

6.6 Test Cases

Test Case ID	Test Scenario	Expected Result	Status
TC01	Register with valid details	Account created	Pass
TC02	Register with duplicate email	Error message displayed	Pass
TC03	Login with valid credentials	Dashboard displayed	Pass
TC04	Login with invalid password	Login failed	Pass
TC05	Search valid stock symbol	Stock details displayed	Pass
TC06	Search invalid stock symbol	Error message displayed	Pass
TC07	Buy stock with sufficient balance	Stock purchased	Pass
TC08	Buy stock with insufficient balance	Purchase blocked	Pass
TC09	Sell owned stock	Portfolio updated	Pass
TC10	Sell unavailable stock	Error displayed	Pass
TC11	View portfolio	Holdings displayed	Pass
TC12	View transaction history	Transactions displayed	Pass
TC13	Update profile	Profile updated	Pass
TC14	Logout	Session terminated	Pass
TC15	Access protected page without	Access denied	Pass

Test Case ID	Test Scenario	Expected Result	Status
--------------	---------------	-----------------	--------

login

6.7 Validation Results

The validation process confirmed that:

- User registration works correctly.
- Password encryption functions properly.
- JWT authentication secures protected routes.
- Portfolio calculations are accurate.
- Transaction history is maintained correctly.
- API responses are successfully retrieved.
- MongoDB stores and retrieves data efficiently.

6.8 Bug Report

During development, the following issues were identified and resolved.

Bug ID	Description	Solution
B01	Duplicate email registration	Added unique email validation
B02	Invalid JWT token	Implemented token verification middleware
B03	Portfolio not updating	Fixed database update logic

Bug ID	Description	Solution
B04	Incorrect balance calculation	Updated trading algorithm
B05	API timeout	Added error handling and retry mechanism

6.9 Performance Evaluation

The performance of the application was evaluated based on responsiveness and resource utilization.

Parameter	Observation
-----------	-------------

Login Response	Fast
----------------	------

Stock Search	Fast
--------------	------

Portfolio Loading	Fast
-------------------	------

Database Queries	Efficient
------------------	-----------

API Response	Acceptable
--------------	------------

Memory Usage	Moderate
--------------	----------

CPU Usage	Low
-----------	-----

The system performed efficiently under normal operating conditions.

6.10 User Acceptance Testing (UAT)

The application was evaluated by a small group of users to verify usability and functionality.

Feedback

- User-friendly interface.
- Simple navigation.
- Fast login process.
- Accurate portfolio calculations.
- Easy stock search.
- Responsive design.

Overall, users expressed satisfaction with the application's functionality and ease of use.

6.11 Testing Summary

The testing phase confirmed that the **SB Stocks – Stock Portfolio Management System** satisfies all functional and non-functional requirements. All major modules passed testing successfully, and identified issues were resolved before deployment. The application demonstrated reliable performance, secure authentication, accurate portfolio management, and effective integration with the Alpha Vantage API.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Introduction

The Results and Discussion chapter presents the successful implementation and evaluation of the **SB Stocks – Stock Portfolio Management System**. This chapter demonstrates the functionality of the developed application through different user interfaces and explains how each module fulfills the project objectives.

The application was successfully developed using the MERN Stack and integrated with the Alpha Vantage API for real-time stock market data. All modules were tested and validated to ensure reliable performance, secure authentication, and accurate portfolio management.

7.2 Home Page

The Home Page serves as the entry point of the application. It introduces users to the platform and provides options to register or log in.

Features

- Attractive user interface
- Navigation bar
- Login button
- Register button
- Project overview
- Responsive layout

Result

The homepage successfully provides users with easy navigation and a professional first impression.

Figure 7.1: Home Page (*Insert Home Page Screenshot Here*)

7.3 User Registration

The registration module allows new users to create an account by entering their name, email address, and password.

Features

- User validation
- Duplicate email prevention
- Password encryption using bcrypt
- Successful account creation

Result

New users can register securely, and account details are stored in MongoDB Atlas.

Figure 7.2: Registration Page (*Insert Screenshot*)

7.4 User Login

The login module authenticates registered users using JWT Authentication.

Features

- Email verification
- Password verification
- Secure JWT token generation
- Protected routes

Result

After successful authentication, users are redirected to the dashboard.

Figure 7.3: Login Page *(Insert Screenshot)*

7.5 Dashboard

The Dashboard provides an overview of the user's investment portfolio.

Information Displayed

- Available Balance
- Total Portfolio Value
- Total Investment
- Number of Stocks
- Recent Transactions

Result

Users can quickly monitor their trading activities and portfolio performance.

Figure 7.4: Dashboard *(Insert Screenshot)*

7.6 Stock Search Module

The Stock Search Module retrieves real-time stock information from the Alpha Vantage API.

Information Displayed

- Company Name
- Stock Symbol
- Current Price
- Market Information

Result

The system successfully fetches stock data and displays it to the user.

Figure 7.5: Stock Search *(Insert Screenshot)*

7.7 Buy Stock Module

The Buy Stock module allows users to purchase stocks using virtual money.

Workflow

1. Search stock.
2. Enter quantity.
3. Validate balance.
4. Purchase stock.
5. Update portfolio.
6. Record transaction.

Result

Stock purchases are processed successfully, and portfolio details are updated automatically.

Figure 7.6: Buy Stock *(Insert Screenshot)*

7.8 Sell Stock Module

The Sell Stock module enables users to sell stocks previously purchased.

Workflow

1. Select owned stock.

2. Enter quantity.
3. Validate quantity.
4. Update portfolio.
5. Record transaction.
6. Update balance.

Result

Users can successfully sell stocks and view updated balances.

Figure 7.7: Sell Stock (*Insert Screenshot*)

7.9 Portfolio Module

The Portfolio module displays all investments made by the user.

Information Displayed

- Company Name
- Stock Symbol
- Quantity
- Purchase Price
- Current Price
- Total Investment
- Profit/Loss

Result

Users can monitor portfolio performance in real time.

Figure 7.8: Portfolio (*Insert Screenshot*)

7.10 Transaction History

The Transaction History module records all buying and selling activities.

Details Displayed

- Stock Name
- Transaction Type
- Quantity
- Price
- Date
- Time

Result

Every transaction is stored permanently and displayed accurately.

Figure 7.9: Transaction History (*Insert Screenshot*)

7.11 User Profile

The Profile module allows users to manage their personal information.

Features

- View profile
- Update name
- Change password
- Update email

Result

Profile details are updated securely and reflected immediately.

Figure 7.10: Profile Page (*Insert Screenshot*)

7.12 API Integration Results

The Alpha Vantage API was successfully integrated with the application.

Successfully Retrieved Data

- Company Information
- Stock Symbol
- Current Market Price
- Daily Market Data

Result

Real-time stock information was displayed without affecting application performance under normal usage.

7.13 Database Results

MongoDB Atlas successfully stored:

- User accounts
- Portfolio details
- Trading history
- Authentication data

Advantages

- Fast retrieval
- Secure storage

- Cloud accessibility
 - High scalability
-

7.14 Security Results

The application successfully implemented:

- JWT Authentication
- Password Encryption
- Protected API Routes
- Secure Login
- Input Validation

Result

No unauthorized access was observed during testing.

7.15 Performance Analysis

Parameter	Result
Login Time	< 2 seconds
Dashboard Loading	< 3 seconds
Portfolio Loading	< 2 seconds
API Response	2–5 seconds (depends on Alpha Vantage API)
Database Operations	Fast
Memory Usage	Moderate

Parameter	Result
-----------	--------

CPU Usage	Low
-----------	-----

The application demonstrated stable performance throughout testing.

7.16 Comparison with Project Objectives

Objective	Status
-----------	--------

Secure Authentication	✓ Achieved
-----------------------	------------

Portfolio Management	✓ Achieved
----------------------	------------

Paper Trading	✓ Achieved
---------------	------------

Transaction History	✓ Achieved
---------------------	------------

Real-Time Stock Data	✓ Achieved
----------------------	------------

Responsive UI	✓ Achieved
---------------	------------

MongoDB Integration	✓ Achieved
---------------------	------------

REST API Development	✓ Achieved
----------------------	------------

All project objectives were successfully met.

7.17 User Feedback

The application was evaluated by students and peers.

Positive Feedback

- Easy to use

- Beginner-friendly interface
- Smooth navigation
- Realistic paper trading experience
- Accurate portfolio calculations
- Secure login process

Suggested Improvements

- Watchlist functionality
- Stock market news integration
- AI-based investment recommendations
- Mobile application support
- Portfolio analytics and charts

7.18 Discussion

The project successfully demonstrates the practical implementation of a modern web-based stock portfolio management system. The MERN architecture enabled efficient communication between the client and server, while MongoDB Atlas ensured reliable cloud-based data storage. JWT authentication provided secure user access, and the Alpha Vantage API delivered live market data for a realistic paper trading experience.

Although the system relies on the free tier of the Alpha Vantage API, which has request limits, it effectively meets the educational objectives of the project. The modular design also allows future enhancements without major architectural changes.

7.19 Chapter Summary

The **SB Stocks – Stock Portfolio Management System** was successfully implemented and tested. The application fulfilled all planned objectives, including secure authentication, paper trading, portfolio management, and transaction tracking. The results demonstrate that the proposed system provides a reliable and practical learning platform for users interested in understanding stock market operations without financial risk.

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Introduction

The conclusion summarizes the overall achievements of the project and evaluates whether the objectives defined during the planning phase have been successfully accomplished. This chapter also discusses the limitations of the current system and highlights possible enhancements that can be implemented in future versions.

The **SB Stocks – Stock Portfolio Management System** was developed as a web-based paper trading platform that enables users to simulate stock market trading using virtual money. The application combines a modern user interface, secure authentication, cloud database integration, and real-time market data to provide a practical and educational stock trading experience.

8.2 Project Summary

The project successfully demonstrates the design and implementation of a secure and responsive stock portfolio management system using

the MERN Stack. The application allows users to register, log in securely, search stocks, simulate stock purchases and sales, manage portfolios, and maintain a complete transaction history.

The React.js frontend provides an intuitive and responsive user interface, while the Express.js backend manages business logic and RESTful API communication. MongoDB Atlas serves as the cloud database for storing user accounts, portfolios, and transaction records. JWT authentication ensures secure access to protected resources, and the Alpha Vantage API supplies real-time stock information.

The application offers a realistic paper trading environment without involving actual financial transactions, making it particularly useful for students, beginners, and individuals who want to learn stock market concepts before investing real money.

8.3 Objectives Achieved

The following objectives were successfully achieved during the development of the project:

- Developed a complete web-based stock portfolio management system.
- Implemented secure user registration and login using JWT authentication.
- Integrated MongoDB Atlas for cloud-based data storage.
- Connected the application with the Alpha Vantage API for real-time stock data.
- Enabled users to buy and sell stocks using virtual currency.
- Maintained accurate portfolio records and transaction history.
- Designed a responsive and user-friendly interface using React.js.

- Built scalable RESTful APIs using Express.js and Node.js.
- Successfully deployed the frontend and backend on cloud platforms.

All planned objectives were completed successfully.

8.4 Advantages of the System

The developed application offers several advantages over traditional learning methods:

- Risk-free paper trading environment.
 - Real-time stock market information.
 - Secure authentication and authorization.
 - Cloud database support.
 - User-friendly interface.
 - Fast response time.
 - Modular and scalable architecture.
 - Complete portfolio tracking.
 - Detailed transaction history.
 - Easy maintenance and future expansion.
-

8.5 Limitations of the Current System

Although the project fulfills its intended objectives, it has certain limitations:

- The application supports only virtual trading and does not execute real stock market transactions.

- Stock data availability depends on the Alpha Vantage API.
- The free API plan has request rate limitations.
- Advanced portfolio analytics and visual charts are not included.
- Stock market news and financial reports are not integrated.
- Multi-user trading competitions are not supported.
- Push notifications and alerts are unavailable.
- The application is currently optimized for web browsers only and does not include a dedicated mobile application.

These limitations provide opportunities for future improvements.

8.6 Future Enhancements

Several advanced features can be incorporated into future versions of the system to improve its functionality and user experience.

1. AI-Based Stock Recommendation System

Machine Learning algorithms can analyze historical stock data and provide personalized investment suggestions based on user preferences and market trends.

2. Watchlist Feature

Users can maintain a personalized watchlist to monitor their favorite stocks and track price movements over time.

3. Interactive Charts

Integrating financial charting libraries such as Chart.js or TradingView can provide users with visual representations of stock performance and portfolio growth.

4. Stock Market News

The application can integrate financial news APIs to display the latest market updates, helping users make informed investment decisions.

5. Portfolio Analytics

Advanced analytics can be added to display:

- Daily profit and loss
- Portfolio diversification
- Sector-wise investments
- Investment trends
- Return on Investment (ROI)

6. Price Alerts and Notifications

Users can receive email or mobile notifications when selected stocks reach predefined price targets.

7. Mobile Application

A dedicated Android and iOS application can be developed using React Native or Flutter to improve accessibility.

8. Multi-User Competitions

Users can participate in virtual trading contests and compare portfolio performance with other participants.

9. Admin Dashboard

An administrative panel can be developed to manage users, monitor application usage, and generate analytical reports.

10. Multi-Language Support

The interface can support multiple languages to improve accessibility for users from different regions.

8.7 Learning Outcomes

The development of this project provided valuable practical experience in several areas of software engineering and web development.

The following skills were gained:

- Full Stack Web Development using the MERN Stack.
- RESTful API development.
- Cloud database management using MongoDB Atlas.
- Secure authentication using JWT and bcrypt.
- Integration of third-party APIs.
- React.js component-based architecture.
- Backend development using Express.js.
- Version control using Git and GitHub.
- Deployment using Vercel and Render.
- Software testing and debugging.

These skills strengthen the understanding of modern web application development and cloud-based software deployment.

8.8 Final Conclusion

The **SB Stocks – Stock Portfolio Management System** successfully fulfills its objective of providing a secure, scalable, and user-friendly platform for paper trading. By integrating modern web technologies with real-time stock market data, the application enables users to gain practical experience in stock trading without financial risk.

The project demonstrates the effective use of the MERN Stack, cloud computing, RESTful APIs, and secure authentication mechanisms. It provides a valuable educational tool for students and beginner investors while also showcasing practical full-stack development skills.

The modular architecture allows future enhancements such as AI-driven recommendations, advanced analytics, mobile support, and stock market news integration, ensuring that the application can evolve into a more comprehensive financial learning platform.

Overall, the project represents a successful implementation of modern web technologies to solve a real-world problem in financial education and portfolio management.

REFERENCES (IEEE FORMAT)

1. Patel, J., Shah, S., Thakkar, P., & Kotecha, K., "Predicting Stock Market Index Using Machine Learning Techniques," *Expert Systems with Applications*, Elsevier.
2. Fama, E. F., "Efficient Capital Markets: A Review of Theory and Empirical Work," *Journal of Finance*, Vol. 25, No. 2.
3. React Documentation. Available: <https://react.dev>
4. Node.js Documentation. Available: <https://nodejs.org>
5. Express.js Documentation. Available: <https://expressjs.com>
6. MongoDB Atlas Documentation. Available: <https://www.mongodb.com/docs/atlas>
7. Alpha Vantage API Documentation. Available: <https://www.alphavantage.co/documentation>
8. JSON Web Token (JWT). Available: <https://jwt.io>
9. OWASP Foundation. Available: <https://owasp.org>

10. GitHub Documentation. Available: <https://docs.github.com>
 11. Visual Studio Code Documentation. Available: <https://code.visualstudio.com/docs>
 12. MDN Web Docs. Available: <https://developer.mozilla.org>
-

APPENDIX A – SOFTWARE REQUIREMENTS

- Operating System: Windows 10/11
 - IDE: Visual Studio Code
 - Frontend: React.js
 - Backend: Node.js, Express.js
 - Database: MongoDB Atlas
 - Authentication: JWT, bcrypt
 - Version Control: Git, GitHub
 - API Testing: Postman
 - Browser: Google Chrome
-

APPENDIX B – HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: 20 GB free disk space
- Internet Connection: Broadband
- Display Resolution: 1366 × 768 or higher

APPENDIX C – USER MANUAL

1. Open the application in a web browser.
2. Register using your name, email, and password.
3. Log in with your registered credentials.
4. Access the dashboard to view your virtual balance and portfolio summary.
5. Search for stocks using the search feature.
6. Buy or sell stocks using virtual funds.
7. View your updated portfolio and transaction history.
8. Update your profile details if required.
9. Log out securely after completing your session.

APPENDIX D – DEPLOYMENT INFORMATION

Frontend: React.js hosted on Vercel

Backend: Node.js & Express.js hosted on Render

Database: MongoDB Atlas (Cloud)

Version Control: GitHub Repository